

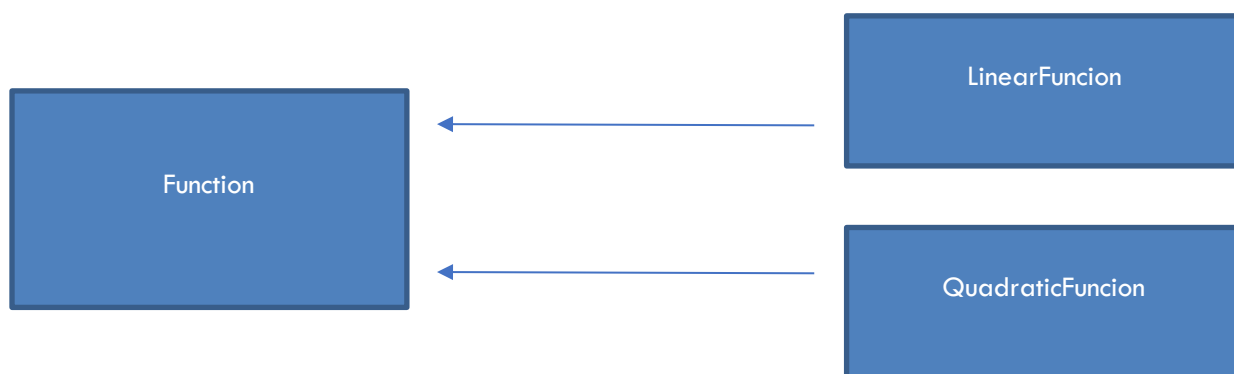
FunctionSolver

Wstęp.....	1
Funkcja	1
Calc().....	1
LinearFunction.....	1
LinearFunction()	1
Calc().....	1
IsPerpendicularTo().....	1
angle().....	1
QuadraticFunction.....	1
QuadraticFunction()	1
Calc().....	1
Vertex().....	1
Intersection()	1

Wstęp

Program służy do obliczania punktów wspólnych funkcji kwadratowej i liniowej. Tworzę 2 klasy potomne, które przez konstruktory przyjmują współczynniki do samych siebie.

Działają na liczbach zmiennoprzecinkowych pojedynczej precyzji (float'y) w celu szybszego obliczania wartości. Więcej liczb przy tym można upakować do rejestrów SIMD (jak XMM, YMM, ZMM), co znacznie przyspiesza obliczenia.



Funkcja

abstract class **Funkcja**

Klasa bazowa dla dowolnej funkcji.

Metody

Calc()

virtual float **Calc(float x)**

Oblicza wartość funkcji

Argumenty:

- x (float) – argument

Wartość zwracana: Wynik wyrażony w typie FP32

LinearFunction

class LinearFunction

Klasa reprezentująca funkcję liniową w postaci kierunkowej. Można obliczyć poza punktami wspólnymi również kąt między nią a osią OX, czy prostopadłość między dwoma funkcjami liniowymi

Metody

LinearFunction()

LinearFunction(float a, float b)

Inicjalizacja funkcji liniowej

Argumenty:

- a, b (float) – współczynniki

Ograniczenia:

$$a \neq 0 \ \&\& \ b \neq 0$$

Wartość zwracana: Obiekt typu *LinearFunction*

Calc()

float Calc(float x)

Oblicza wartość funkcji

Argumenty:

- x (float) – argument

Wartość zwracana: Wynik wyrażony w typie FP32

IsPerpendicularTo()

bool isPerpendicularTo(LinearFunction _f2)

Sprawdza prostopadłość dwóch funkcji liniowych (prostych). Korzysta z zależności:

$$f(x) \perp g(x) \Rightarrow af * ag = -1$$

Argumenty:

- _f2 (LinearFunction) – inna funkcja liniowa

Wartość zwracana: *true* dla prostopadłych, *false* dla nieprostopadłych

angle()

float angle(bool returnInRadians = false)

Oblicza kąt między osią OX, a samą funkcją (inaczej argument). Kąt jest wartością z przedziału $[0^\circ, 360^\circ]$ lub $[0, 2\pi]$ przy zwracaniu w radianach

Argumenty:

- `returnInRadians (bool)` – czy zwrócić w radianach

Wartość zwracana: domyślnie wartość zwracana w stopniach, przy ustawieniu argumentu funkcji na *true* w radianach

`(float?, float?) / float?` **Intersection(<Klasa > _funcB)**

Oblicza punkty wspólne funkcji z inną funkcją.

Argumenty:

- `_funcB (LinearFunction/QuadraticFunction)` – druga funkcja

Wartość zwracana:

Kolejno:

- `(null, null) / null` – dla braku rozwiązań z funkcją kwadratową / liniową
- `(float, null) / float` – dla jednego rozwiązania z funkcją kwadratową / liniową
- `(float, float)` – dla dwóch rozwiązań z funkcją kwadratową

`QuadraticFunction`

class **QuadraticFunction**

Klasa reprezentująca funkcję kwadratową w postaci ogólnej. Automatycznie przy inicjalizacji oblicza wierzchołek funkcji

Metody

QuadraticFunction()

QuadraticFunction(float a, float b, float c)

Inicjalizacja funkcji liniowej

Argumenty:

- a, b, c (float) – współczynniki

Ograniczenia:

$$a \neq 0$$

Wartość zwracana: Obiekt typu *QuadraticFunction*

Calc()

float Calc(float x)

Oblicza wartość funkcji

Argumenty:

- x (float) – argument

Wartość zwracana: Wynik wyrażony w typie FP32

Vertex()

(float, float) Vertex()

Zwraca współrzędne wierzchołka paraboli

Wartość zwracana: Krotka 2-wartościowa oznaczająca kolejno argument i wartość funkcji przy niej (p i q)

Intersection()

float?, float?) / float? Intersection(<Klasa > _funcB)

Oblicza punkty wspólne funkcji z inną funkcją.

Jeśli chodzi o sposób obliczania punktów wspólnych, jest dokonane po prostu przez przyrównanie dwóch równań parabol.

$$ax^2 + bx + c = dx^2 + ex + f$$

$$(a - d)x^2 + (b - e)x + (c - f) = 0$$

Dla uproszczenia:

$$a - d = a'$$

$$b - e = b'$$

$$c - f = c'$$

Wzór na wyróżnik funkcji kwadratowej wtedy będzie wyglądał tak:

$$\Delta = b'^2 - 4a'c'$$

Zatem:

$$\Delta = (b - e)^2 - 4 \cdot (a - d)(c - f)$$

$$\Delta = b^2 - 2be + e^2 - 4 \cdot (ac - dc - af + df)$$

A wiedząc że liczba rozwiązań jest skorelowana z wartością wyróżnika, więc wartością zwracaną będzie tutaj jedna z 3 możliwości:

- (null, null), $\Delta < 0$
- (float, null), $\Delta = 0$
- (float, float), $\Delta > 0$

Podobnie w taki sposób można wyszukać punktów wspólnych z funkcją liniową.

Argumenty:

- `_funcB (LinearFunction/QuadraticFunction)` – druga funkcja

Wartość zwracana:

Kolejno:

- (null, null) / null – dla braku rozwiązań z funkcją kwadratową / liniową
- (float, null) / float – dla jednego rozwiązania z funkcją kwadratową / liniową
- (float, float) – dla dwóch rozwiązań z funkcją kwadratową

